算法基础知识点梳理

目录

第一章 算法复杂度 2

1.1 时间复杂度的含义 2

1.2 渐近记号与空间开销 2

第二章 查找与分治 2

2.1 二分查找 2

2.2 归并排序 2

第一章 算法复杂度

1.1 时间复杂度的含义

时间复杂度描述输入规模增大时，算法基本操作次数的增长规律。它关注增长趋势，而不是某台计算机上的实际运行秒数。分析前应明确输入规模、基本操作和讨论的情形。

最坏情况与平均情况是不同的分析对象。最坏情况给出同一规模输入中的最大操作次数；平均情况需要对输入分布作出假设。不能省略假设后直接把平均复杂度当作普遍结论。

1.2 渐近记号与空间开销

大 O 表示渐近上界，Ω 表示渐近下界，Θ 表示渐近紧确界。将低次项与常数因子忽略，适用于比较规模较大时的增长规律；规模较小时，常数、访存与实现方式也可能影响实际速度。

$$T(n) = 3n^2 + 2n + 1 = \Theta(n^2)$$

空间复杂度衡量随输入规模增长所需空间的变化。讨论辅助空间时，应区分输入数据本身与算法额外使用的存储；递归调用栈也属于需要考虑的空间。

增长量级	增长特点	常见机制
Θ(1)	不随输入规模增长	固定次数的访问或运算
Θ(log n)	输入范围按固定比例缩小	二分查找
Θ(n)	工作量与规模成正比	遍历全部元素
Θ(n log n)	分治层数与每层总工作量共同决定	标准归并排序

第二章 查找与分治

2.1 二分查找

二分查找要求数据有序，并能够高效访问中间位置。每次将目标值与中间元素比较，根据大小关系排除不可能包含目标的一半范围，直到找到目标或区间为空。

有序性保证排除半区的正确性。若数组未排序，比较中间元素不能判断目标在哪一边。若使用不支持高效随机访问的数据结构，也不能直接照搬数组上每次定位中点的常数开销假设。

$$T(n) = T(\lfloor n/2 \rfloor) + \Theta(1) = \Theta(\log n)$$

迭代实现的辅助空间为 Θ(1)；递归实现通常需要 Θ(log n) 的调用栈。实现时需统一左右边界的包含规则，并保证区间在每次失败比较后缩小，避免死循环。

2.2 归并排序

归并排序将序列分为两个较小子序列，递归地排序，再将两个有序序列合并。划分建立子问题，合并将局部有序结果组合为整体有序结果。

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$$

对按元素比较的标准数组实现，每层合并的总工作量为 Θ(n)，递归层数为 Θ(log n)，因此总时间为 Θ(n log n)。通常需要 Θ(n) 的辅助数组；合并相等元素时优先取左侧元素，可以保持稳定性。